



Operations → AI/ML Engineer, in 90 days

A week-by-week route from where you are to a portfolio and resume that can credibly compete for AI/ML Engineer roles — built around the hours and budget you actually have, leaning hard on the LLM-application skills you've already started building.

CURRENT ROLE	TARGET ROLE	RUNWAY	BUDGET
Ops / Support, 1–3 yrs	AI/ML Engineer	12–20 hrs / week	Free resources only

1. The 90 days at a glance

One continuous project — a support-ticket AI system — gets more capable each month. Skim this, then read the detail in the sections below.

MONTH 1 Foundations KEY SKILLS Python fluency Linear algebra & stats Classical ML PROJECT SLA-breach risk classifier MILESTONE Comfortable training and evaluating a model; can read a confusion matrix, not just accuracy.	MONTH 2 Depth KEY SKILLS Deep learning basics FastAPI Docker PROJECT Deploy the classifier as a live API MILESTONE Shipped a model as a callable endpoint — engineering, not just a notebook.	MONTH 3 Differentiation KEY SKILLS RAG & embeddings Claude API engineering Interviewing PROJECT RAG-powered support assistant MILESTONE
---	---	--

Three shipped projects on GitHub telling one story; applications in flight.

2. Skill gap analysis

Ranked by how often it actually gates an interview, not by how interesting it is to learn.

#	SKILL	STATUS
1	Python programming fluency — functions, OOP, real debugging	Build
2	Data structures & algorithms — enough for technical screens	Build
3	Math foundations — linear algebra, probability & statistics	Build
4	Classical ML (scikit-learn) — regression, classification, evaluation, overfitting	Build
5	Deep learning fundamentals — neural nets, PyTorch basics	Build
6	LLM/GenAI engineering — prompt engineering, embeddings, RAG	Already building
7	Model deployment / MLOps basics — FastAPI, Docker, Hugging Face Spaces	Build
8	SQL & data handling (pandas)	Build
9	Git / version control	Have
10	Stakeholder communication, cross-functional collaboration	Have
11	Software engineering practices — testing, clean code, debugging discipline	Build (light)

3. The 90 days, in detail

Twelve weeks, roughly 15 hrs/week. Each month ends with a shipped, working piece of the same project — that's deliberate, not a stretch goal.

MONTH 01 Foundations — Python, math, classical ML

WEEKS 1–2

Python for engineers, not just scripts

freeCodeCamp: Python for Everybody (Dr. Chuck, YouTube, free) for real fluency; daily reps on **Exercism** / **Codewars** (free) to build debugging discipline, not just syntax recognition.

WEEKS 3–4

Math intuition + classical ML

3Blue1Brown: Essence of Linear Algebra (YouTube) + **StatQuest** for probability/stats intuition; **Kaggle Learn: Intro to Machine Learning** + **Intermediate Machine Learning** micro-courses (free, hands-on scikit-learn).

 **Capstone: Project 1 — SLA-breach risk classifier** (see §4)

MONTH 02 Depth — deep learning, shipping a model

WEEKS 5–6

Deep learning fundamentals

fast.ai: Practical Deep Learning for Coders, Part 1 (free, project-first) alongside **3Blue1Brown's neural network series** for the intuition fast.ai assumes you already have.

WEEKS 7–8

Serving a model like an engineer, not a notebook

The official **FastAPI tutorial** (free docs) to wrap a model as an API; **freeCodeCamp: Docker for Beginners** (YouTube) to containerize it; deploy free on **Hugging Face Spaces** or Render's free tier.

 **Capstone: Project 2 — Deploy the classifier as a live API** (see §4)

MONTH 03 Differentiation — LLM engineering, resume, applying

WEEKS 9–10

Applied GenAI engineering

Anthropic's own **Claude Cookbook** (github.com/anthropics/anthropic-cookbook, free) — directly on the stack you already know; **DeepLearning.AI's** free short courses "Building Systems with the ChatGPT API" and "LangChain for LLM Application Development" for RAG patterns and vector databases.

WEEKS 11–12

Resume, portfolio, and applying

Draft resume + LinkedIn using the STAR method on your ops-role stories, reframed for AI/ML engineering (see §5). Start applying from **week 10** — don't wait for week 12 to feel "ready."

 **Capstone: Project 3 — RAG-powered support assistant (see §4)**

4. Portfolio projects

Three projects, one continuous system — a support-ticket AI pipeline that gets more capable each month, so the portfolio reads as a build, not three disconnected exercises.

1 "Will this ticket breach SLA?" classifier

Train a scikit-learn classification model on support-ticket data (Kaggle's "Customer Support Ticket Dataset," or data shaped like what you see at work) predicting SLA-breach risk from ticket features. Evaluate with precision/recall, not just accuracy.

Python pandas scikit-learn

Resume line — "Built a classification model (scikit-learn) predicting SLA-breach risk from support-ticket data, reaching X% precision — automating a judgment call currently made manually."

2 Ship it as a live API

Wrap the trained classifier in a FastAPI service, containerize it with Docker, and deploy it free on Hugging Face Spaces. Add basic request/response logging so it behaves like a real service, not a demo.

FastAPI Docker Hugging Face Spaces

Resume line — "Deployed a trained ML model as a containerized FastAPI service on Hugging Face Spaces — end-to-end from notebook to a callable endpoint, not just a Jupyter demo."

3 RAG-powered support assistant

Combine the classifier's risk flag with a vector store of historically resolved tickets and the Claude API, so a support rep gets a suggested resolution pulled from similar past tickets instead of starting from scratch.

Resume line — "Built a RAG-based support assistant (Claude API + vector search) that surfaces similar resolved tickets in real time — the applied LLM-engineering project most entry-level ML candidates won't have."

5. Resume bridge — reframing your ops experience

You're not starting from zero. These are real, honest translations of what the ops/support role already taught you, in language an AI/ML engineering hiring manager recognizes.

Tracking resolution-time & SLA metrics, finding the bottleneck

→

The root-cause instinct engineers use when a model's predictions silently degrade in production

Keeping ticket records accurately tagged and categorized

→

The "garbage in, garbage out" data-integrity discipline that matters more in ML than almost anywhere else

Building recurring reports for leadership review

→

The same structured, repeatable-process mindset a model pipeline needs to run correctly, unattended

Flagging a ticket-tag trend and partnering with product/eng on a fix

→

The exact loop an ML engineer runs: "the data flags X" → "the team ships a fix"

Handling escalations against a fixed SLA clock

→

The on-call discipline of keeping a production ML system healthy when downtime has a clock on it too

6. Timeline & milestones



END OF MONTH 1 · WEEK 4

Comfortable writing and debugging real Python. Can explain bias/variance and



END OF MONTH 2 · WEEK 8

Comfortable training and evaluating a model — and shipping it as a live endpoint, not just a notebook. Project



END OF MONTH 3 · WEEK 12

Project 3 shipped. All three projects on GitHub telling one continuous story.

read a confusion matrix.
Project 1 shipped.

2 shipped. Resume draft
exists.

Resume/LinkedIn finalized.
Applications in flight since
week 10.

HONEST READ

90 days makes you a credible candidate for applied **AI Engineer** roles — ones built around integrating LLMs and pretrained models into products, exactly what your existing Claude API projects already demonstrate. It does not make you competitive for roles that require training deep learning models from scratch or a strong CS/math background — those typically need 6–12+ months more study. Budget another 4–8 weeks of active interviewing on top either way. Realistic total timeline to an offer: **5–7 months** — longer than a pure data-analyst path, because the technical bar is higher.

Built for the switch from Ops/Support to AI/ML Engineer · revisit this weekly, not daily